FREE WORKSHOP · JULY 18 · 11 AM

# Advanced Quantum Algorithms

## Beyond QSVM and QAOA: exploring the next frontier of quantum computing

**Nikhil Londhe**
Senior Researcher, Bloq

bloq

# Why “beyond QSVM and QAOA”?

- Every quantum computing tutorial ends at the same two places: a kernel SVM and a MaxCut QAOA.
- The interesting question is what the **last three years** of research produced, and whether any of it is runnable today.
- Today: **four algorithms from recent literature**, each explained in a few minutes and then **run live**.

**The rule for the next hour:** nothing is a slide-only claim. If we can’t run it in front of you, we’ll say so. Exactly one algorithm today has that caveat.

# The next 60 minutes

| # | Algorithm | One-line pitch |
|---|---|---|
| 1 | **PQK** · Projected Quantum Kernels | Quantum kernels that dodge the concentration problem |
| 2 | **QRC** · Quantum Reservoir Computing | Time-series forecasting with **zero** circuit training |
| 3 | **QeMCMC** · Quantum-enhanced MCMC | Quantum proposals inside a classical Metropolis loop |
| 4 | **QKAN** · Quantum Kolmogorov-Arnold Networks | Learnable activation functions, built from QSVT |

Every demo runs on the **Bloq platform**.

SEGMENT 1

# Projected Quantum Kernels

Huang et al., “Power of data in quantum machine learning,” Nature Communications 12, 2631 (2021)

# Introduction

A quantum circuit embeds each data point, then the state is **projected back to classical space** through single-qubit reduced density matrices, which avoids the exponential concentration that kills raw quantum kernels. An RBF kernel on those projections feeds a standard classical SVM.

$$\rho_k(x) = \frac{I + \langle X_k \rangle X + \langle Y_k \rangle Y + \langle Z_k \rangle Z}{2}$$

$$k^{\mathrm{PQ}}(x_i, x_j) = \exp\left( -\gamma \sum_k \left\| \rho_k(x_i) - \rho_k(x_j) \right\|_F^2 \right)$$

Huang et al., “Power of data in quantum machine learning,” Nature Communications 12, 2631 (2021)

# Trade-offs & best practices

### ✓ Advantages

- No kernel concentration: generalizes where fidelity kernels flatline
- $O(N)$ circuit runs (one per point), not $O(N^2)$ fidelity tests
- Nothing variational: no gradients, no barren plateaus; the SVM step is convex

### ✗ Disadvantages

- Quantum advantage is **dataset-dependent**; a classical RBF often ties it
- Shot noise perturbs the kernel matrix; hardware runs need high shot counts
- Feature-map choice is heuristic, with no principled recipe

**Best practices:** prototype on `statevector` (exact, fast) and add shots only for hardware · start with `reps = 1` and deepen cautiously · PCA features down to the qubit count · tune $\gamma$ and $C$ with a **classical** grid search: the kernel is classical after projection, so cross-validation is free.

# Parameters & how to tune them

| Parameter | What it controls | How to tune |
|---|---|---|
| `Qubits` | Circuit width; PCA maps features onto it | Match the feature count; 4–8 is the sweet spot |
| `Feature Map` | The embedding circuit | Start `E2`; try `ZZ` / `Pauli`; plug in your own saved circuit |
| `Circuit Depth` | Feature-map repetitions (expressivity) | 1 first; 2+ only if underfitting, since depth adds noise on hardware |
| `Gamma` | RBF bandwidth in projected space | Log grid $10^{-3}…10^{1}$, cross-validated classically |
| `C` | SVM regularization | Log grid $10^{-2}…10^{2}$, same CV loop |
| `Shots` | Sampling per expectation value | `statevector` while prototyping; ≥ 1024 on hardware |

Only the top three touch the quantum computer: $\gamma$, $C$, and the CV loop are free classical post-processing.

# What can you build with it?

### Task types

- **Binary classification**, today's demo
- **Multiclass**, handled one-vs-one by the SVM automatically
- **Regression**, same kernel with an SVR readout
- **Anomaly detection**, one-class SVM on the projected kernel

### Where it shines / where it doesn't

- Small-to-medium **tabular** datasets (≈ $10^2$–$10^3$ samples)
- Features reducible to the qubit count via PCA
- Not for large datasets ($N^2$ kernel) or raw image / text data

Now, the same algorithm as a **form on the Bloq platform.**

# PQK, live

● LIVE DEMO

**What we'll run:**

- Binary classification, 8-feature dataset
- 4 qubits · `E2` feature map (IQP-type) · 1024 shots
- RBF on 1-RDMs → SVM · `aer_simulator`

**Watch for:**

- Test accuracy vs. a classical RBF-SVM on the same split
- Swapping the feature map from a dropdown, including **your own saved circuits**

SEGMENT 2

# Quantum Reservoir Computing

Forecasting time series with a fixed, untrained quantum circuit

# Introduction

A **fixed, untrained quantum circuit** acts as a high-dimensional nonlinear reservoir: each window of past values is encoded as rotation angles, adjacent qubits are entangled, and the Z expectation of every qubit becomes a feature. The only trained component is a classical linear regression on those features.

$$h(x_t) = (\langle Z_1 \rangle, \langle Z_2 \rangle, ..., \langle Z_n \rangle), \quad \hat{y}_{t+1} = w^\top h(x_t) + b$$

At forecast time the circuit is rebuilt recursively, mixing true history with its own predictions.

# Trade-offs & best practices

### ✓ Advantages

- No circuit training at all: no gradients, no barren plateaus; fitting is one least-squares solve
- Entanglement gives rich nonlinear temporal features for free
- Cheap to run: one circuit per time window

### ✗ Disadvantages

- The reservoir is random; feature quality varies and a tuned classical model (ARIMA, ESN) can win
- Recursive forecasting compounds errors as the horizon grows
- Shot noise lands directly in the feature matrix, so the regression inherits it

**Best practices:** normalize the series to $[0, 1]$ before encoding (the values become rotation angles) · start with 5 qubits and a window of 6, then grow the reservoir · keep the forecast horizon short · always run the built-in ARIMA baseline on the same split · use `statevector` while prototyping.

# Parameters & how to tune them

| Parameter | What it controls | How to tune |
|---|---|---|
| `Qubits` | Reservoir width, i.e. the feature dimension | More qubits = richer features; start at 5, grow if underfitting |
| `Time Window` | How many past steps feed each prediction | Match the dominant period; 6 works for smooth signals |
| `Horizon` | How many future steps to predict | Keep small (1–3); error compounds recursively |
| `Device` | Where the reservoir circuit runs | `aer_simulator` for iteration; switch the dropdown for hardware |
| `Shots` | Sampling per expectation value | `statevector` while prototyping; noise hits features directly |

The linear readout has no knobs at all: refitting it is instant, so parameter sweeps are cheap.

# What can you build with it?

### Task types

- **Time-series forecasting**, today's demo
- **Multi-step prediction** via the recursive rebuild
- **Anomaly detection**, flagging points that deviate from the forecast
- Stress test: **chaotic series** like the Lorenz attractor

### Where it shines / where it doesn't

- Univariate, normalized series with a few hundred points
- Periodic or quasi-periodic signals (seasonal data, sensor traces)
- Not for long-horizon extrapolation or noise-dominated data

Let's forecast something live on the platform.

# QRC, live

● LIVE DEMO

**What we'll run:**

- Sine-wave series · 5-qubit reservoir
- History window 6 · forecast horizon 2
- Linear readout · `aer_simulator`

**Watch for:**

- Train and test **$R^2$** on the forecast
- The **ARIMA classical baseline** running side-by-side, same data, same split

SEGMENT 3

# Quantum-enhanced MCMC

Quantum proposals inside classical Markov Chain Monte Carlo, for combinatorial optimization

# Introduction

A hybrid sampler for Ising and QUBO problems: a **warm-started QAOA circuit proposes candidate bitstrings** inside a classical Metropolis-Hastings loop, with replicas at several temperatures exchanging states (parallel tempering). The quantum computer makes the moves smarter; the classical acceptance rule keeps the convergence guarantees.

$$A(s \to s') = \min\left(1, e^{-(E(s')-E(s))/T}\right)$$

Each proposal: run depth-2 QAOA seeded on the chain's current state, sample, keep the top-$k$ lowest-energy candidates, pick one.

# Trade-offs & best practices

## ✓ Advantages

- Quantum proposals jump between distant low-energy basins that random walks can't reach
- The Metropolis rule is untouched, so MCMC convergence guarantees stay intact
- Parallel tempering shakes chains out of local minima on rugged landscapes

## ✗ Disadvantages

- Circuit-hungry: one QAOA execution per chain, per iteration; cost grows fast
- A heuristic with no proven speedup; on easy landscapes classical MCMC ties it
- Fixed shallow QAOA angles ($\gamma$, $\beta$) may produce near-random proposals on some instances

**Best practices:** span the temperature ladder geometrically, $T_{\text{low}}$ near the smallest energy gaps and $T_{\text{high}}$ near the full energy scale · watch per-replica acceptance rates (aim for 20–40%) · start with problems under 10 variables on the simulator · lower `Top K` for greedier proposals, raise it for more exploration.

# Parameters & how to tune them

| Parameter | What it controls | How to tune |
|---|---|---|
| `Replicas` | Parallel chains / temperature rungs | 4 is a solid default; add rungs for rugged landscapes |
| `Low / High Temp` | Ends of the tempering ladder | Low ≈ 0.01 hunts the optimum; high ≈ the energy scale |
| `Gamma / Beta` | QAOA proposal angles (depth 2) | Defaults work; tune only if acceptance collapses |
| `Iterations` | MCMC steps per replica | 100–500; watch the energy trace flatten |
| `Top K` | Low-energy samples kept per proposal | Small = greedy, large = exploratory |
| `Swap Interval` | Steps between replica-exchange attempts | ≈ 10; too frequent wastes proposals |
| `Shots` | Samples per proposal circuit | Enough to fill `Top K`; 100+ is plenty |

# What can you build with it?

### Task types

- **QUBO / Ising optimization**, today's demo
- **MaxCut, scheduling, portfolio selection**, anything you can write as our shared cost-function format
- **Boltzmann sampling**: it is still MCMC, so thermal states come free

### Where it shines / where it doesn't

- Rugged energy landscapes where simulated annealing stalls
- Problems up to ≈ 20 variables on today's simulators
- Not for convex or lightly-frustrated problems; classical solvers win those outright

# QeMCMC, live

● LIVE DEMO

**What we'll run:**

- 6-variable Ising problem
- 4 replicas · $T$ from 0.01 to 2.0
- Warm-started QAOA proposals, top-$k$ filtering

**Watch for:**

- The energy trace of the coldest replica converging to the optimum
- Replica-exchange events kicking a stuck chain loose

SEGMENT 4

# Quantum Kolmogorov-Arnold Networks

Ivashkov et al. (2024), arXiv:2410.04435. A glimpse past the NISQ horizon

# Introduction

KANs flip the neural-network recipe: the **learnable functions sit on edges**, not nodes. QKAN builds each edge activation as a Chebyshev expansion and evaluates all the polynomials **inside the quantum computer**, using Quantum Singular Value Transformation on block-encoded inputs.

$$\varphi_{pq}(x) = \frac{1}{d+1} \sum_{r=0}^{d} w_{pq}^{(r)} T_r(x), \quad T_r(x) = \cos(r \arccos(x))$$

One layer is five block-encoding steps: dilate → Chebyshev → multiply → combine → collapse.

# Trade-offs & best practices

### ✓ Advantages

- Parameter-efficient: one Chebyshev weight vector per edge, and the learned activations are inspectable
- Chebyshev evaluation via QSVT is exact, not sampled from a variational ansatz
- Built on block encodings, so it scales **with** fault-tolerant hardware rather than against it

### ✗ Disadvantages

- **The honest one: fault-tolerant only.** QSVT depth means no current QPU can run it; today it lives on statevector simulation, including ours
- Simulation caps the practical network size
- Gradient-free training (SPSA) on simulated outputs converges slowly

**Best practices:** keep the Chebyshev degree at 3–5, higher degrees oscillate · start with one small hidden layer · always use the `statevector` method · treat it as a research preview: the question it answers is “what will we run when the hardware arrives?”

# Parameters & how to tune them

| Parameter | What it controls | How to tune |
|---|---|---|
| `Hidden Nodes` | Width of the QKAN layer | 4 is plenty for small tabular data; width multiplies qubit cost |
| `Chebyshev Degree` | Polynomial order of each edge activation | 3 by default; raise only for wiggly targets |
| `Loss Function` | Training objective | `Cross Entropy` for classification, `Squared Error` for regression |
| `Optimizer` | How weights update | `ADAM` converges faster; `SPSA` uses fewer evaluations |
| `Epochs` | Training passes | 10 to start; watch the loss curve, not the clock |
| `Learning Rate` | Step size (plus SPSA perturbation) | Small (≈ 0.01); the loss landscape is smooth but shallow |

# What can you build with it?

### Task types

- **Binary classification**, today's demo
- **Multiclass classification** with a wider output layer
- **Nonlinear regression** and smooth function fitting, the classic KAN habitat

### Where it shines / where it doesn't

- Small tabular datasets and low-dimensional function approximation
- Research settings: comparing against classical KANs and MLPs
- Not on real QPUs yet, by construction; simulator only

We'll still run it live, on `statevector`, in the same form you've now seen three times.

# QKAN, live (on statevector)

● LIVE DEMO

**What we'll run:**

- Binary classification, same dataset as the PQK demo for a fair comparison
- 4 hidden nodes · Chebyshev degree 3
- `ADAM` · 10 epochs · `statevector`

**Watch for:**

- The training loss falling epoch by epoch
- Test accuracy vs. this morning's PQK number

THE PLATFORM

# What you just used, four times

## One dropdown to real hardware

Re-open any experiment, switch `aer_simulator` → `ibm_brisbane`. QPU time estimated **before** you submit.

## Your custom circuits

Save a custom feature map once; it appears in every compatible algorithm's dropdown automatically.

## A full algorithm catalog

Beyond today's four: QAOA, VQE, QSVC, QNN, QCNN, QGAN, Ising machines, and more.

THE PLATFORM

# Plans, when you're ready

## Standard

For individuals starting their quantum journey.

**$30** / month

- ✓ 50 experiments per month
- ✓ 120 vCPU hours runtime
- ✓ **4 vCPU, 8 GB RAM** compute
- ✓ 10 notebooks, 10 circuits
- ✓ $10 AI credits (one-time)

Start 14-day free trial

## Pro

For researchers running heavy simulations.

**$100** / month

- ✓ 100 experiments per month
- ✓ 400 vCPU hours runtime
- ✓ **8 vCPU, 16 GB RAM** compute
- ✓ 50 notebooks, 50 circuits
- ✓ $10 AI credits, monthly

Upgrade to Pro

## Teams & Academia

For labs and departments collaborating on research.

**Custom**

- ✓ Compute sized to the need
- ✓ Clusters for 30+ qubit sims
- ✓ Shared team notebooks
- ✓ Add-ons: SSO, dedicated HW

Contact sales

## Enterprise

For organizations deploying production quantum pipelines.

**Custom**

- ✓ Compute sized to the need
- ✓ Custom usage & limits
- ✓ SSO & advanced security
- ✓ SLAs & 24/7 support

Book a demo

# Questions?



Sign up for the platform

**Nikhil Londhe**
Senior Researcher
nikhil.l@bloq.in

bloq